

Examen Réparti 2 PSCR Master 1 Informatique Jan 2024

UE MU4IN400

Année 2023-2024

2 heures – **Tout document papier autorisé sauf corrigés fournis de l'UE imprimés (annales, TD, TME)**
Tout appareil de communication électronique interdit (téléphones...)

Introduction

- Le barème est sur 23 points et est donné à titre indicatif et susceptible d'être modifié.
- Dans le code C++ demandé vous vous efforcerez d'écrire du code correct et compilable, mais les petites typos ne seront pas sanctionnées.
- De même les problèmes d'include et de namespace ne sont pas pertinents (pas la peine de qualifier les `std::`, ni de citer tous les `#include`)
- Sauf indication contraire dans la question, on suppose également que **tous les appels système se passent bien**, on ne demande pas de contrôler les valeurs de retour (donc pas de `perror` ou `errno.h`).
- On vous fournit une annexe recensant la signature de certains appels système.

Le sujet est composé de sous sections indépendantes qu'on pourra traiter dans l'ordre qu'on souhaite.

I. Notions de File descriptor (6 points)

Question 1. (2,5 points) Implémentez la fonction `int copy(int fddest, int fdsrc)`. Cette fonction doit utiliser les appels système `read` et `write` pour transférer le contenu de `fdsrc` vers `fddest`. La fonction doit retourner le nombre total d'octets copiés, ou une valeur négative en cas d'erreur. Assurez la gestion des erreurs pour `read` et `write`, en affichant des messages appropriés avec `perror` et en retournant une valeur négative en cas d'échec. Les opérations de lecture et d'écriture doivent traiter les données par blocs de 1024 octets, sauf pour le dernier bloc.

On rappelle que `read` comme `write` rendent en général le nombre d'octets traités, zéro pour indiquer une fin de fichier, et une valeur négative traduisant une erreur décrite plus précisément à l'aide de `errno`.

Question 2. (1,5 points) Ecrire un programme principal qui simule une version simplifiée de la commande "cat". Si le programme est invoqué sans arguments, il lit sur son entrée standard et copie sur sa sortie standard (`STDIN_FILENO`, `STDOUT_FILENO`). Si des arguments sont fournis, il copie sur la sortie standard chacun des fichiers argument dans l'ordre.

Vous utiliserez la primitive "copy" de la question précédente, qu'on peut supposer résolue. Le programme doit se terminer et rendre `EXIT_FAILURE` si une copie échoue, sinon il rend `EXIT_SUCCESS`.

Question 3. (2 points) Pour chacune des situations suivantes, décrivez comment obtenir les descripteurs de fichier (file descriptors) adaptés à l'utilisation avec votre fonction `copy` implémentée précédemment. Précisez les appels système nécessaires pour créer la ressource correspondante et

obtenir un descripteur de fichier valide pour `copy`. Vous préciserez si le descripteur obtenu peut être passé comme source ou destination de `copy`. Expliquez étape par étape les actions à réaliser.

1. Avec un tube nommé `“./montube”`
2. Avec un tube anonyme
3. Avec une socket TCP, côté client
4. Avec une socket TCP, côté serveur.

II. Signaux (3 points)

Question 4. (3 points) Écrivez un programme C qui implémente le comportement suivant avec deux processus, un père et un fils :

- Le fils doit envoyer un signal SIGINT à son père puis se terminer
- Le père doit masquer le signal SIGINT, attendre la terminaison du fils, puis attendre de recevoir le signal SIGINT à l'aide de `sigsuspend` (ce qui va le terminer).

III. Questions sur Sockets (2,5 points)

Question 5. (2,5 points) Répondez concisément et précisément à chacune des questions suivantes.

1. Dans quel(s) contexte(s) et dans quel but peut on croiser la ligne `bind(sockfd, (struct sockaddr *)&addr, sizeof(addr))` ?
2. Expliquez pourquoi dans cette invocations on a un cast `“(struct sockaddr *)”`. De quel(s) type(s) pourrait être `“addr”` en pratique ?
3. Comment fermer proprement une socket TCP après avoir fini de communiquer ?
4. Quels sont les étapes (appels à l'api) nécessaires pour émettre un paquet UDP vers une adresse de multicast ?
5. A l'aide de quel(s) invocations résoudre le nom du serveur `“www.lip6.fr”` vers une adresse IP utilisable dans l'api des sockets ? Sur quel système distribué sous jacent s'appuie cette invocation ?

IV. Producteur Consommateur en Mémoire partagée (6 points)

Question 6. (2,5 points) Ecrire un programme “producteur” qui doit :

1. Créer et allouer dans `“/myshm”` un segment de mémoire partagée de 4096 octets.
2. Créer et allouer dans `“/mysem”` un sémaphore nommé initialement à zéro.
3. Attendre une saisie de l'utilisateur sur l'entrée standard. On utilisera simplement la primitive `read` sur `STDIN_FILENO`.
4. S'assurer de placer la saisie de l'utilisateur dans le segment de mémoire partagée
5. Déverrouiller le sémaphore pour indiquer que la donnée est prête
6. Se terminer proprement (mais sans détruire les IPC créés).

Question 7. (2 points) Ecrire un programme “consommateur” qui doit :

1. Accéder aux ressources `“/myshm”` et `“/mysem”` créés par l'autre processus
2. Attendre que le producteur ait indiqué que la donnée est disponible dans la mémoire partagée
3. Recopier la saisie de l'utilisateur sur la sortie standard
4. Se terminer proprement en effaçant les IPC créés.

Question 8. (1,5 point) Sans le programmer entièrement, expliquez ce qu'il faudrait modifier dans ces deux programmes pour que le sémaphore soit hébergé dans le segment de mémoire partagée désigné par `“/myshm”`.

V. Producteur Consommateur en Multi-Thread (5,5 points)

On suppose le fichier suivant, à compléter.

mt.cpp

```
1  #include <iostream>
2  #include <thread>
3  #include <mutex>
4  #include <condition_variable>
5  #include <queue>
6
7  std::mutex mtx;
8  std::condition_variable cv;
9  std::queue<int> buffer;
10 const unsigned int maxBufferSize = 10;
11
12 // Producer function
13 void producer(int count) {
14 }
15
16 // Consumer function
17 void consumer() {
18 }
19
20 int main() {
21     return 0;
22 }
```

Question 9. (2,25 points) Compléter la fonction “void producer (int count)” qui insère un par un les entiers de 0 à $count - 1$ dans la queue, en s’assurant que la taille de la queue n’excède jamais “maxBufferSize”. On rappelle qu’on peut insérer un entier i dans la queue avec “buffer.push(i)” et que le remplissage de la queue est donné par “buffer.size()”.

Question 10. (2,25 points) Compléter la fonction “void consumer()” qui consomme un par un les éléments de la queue dans une boucle infinie, en attendant aux moments opportuns qu’il y en ait à lire. On rappelle qu’on peut retirer un entier i dans la queue avec “ $i = \text{buffer.front()}; \text{buffer.pop()};$ ” quand la queue est non vide.

Question 11. (1 points) Compléter le main, qui doit lancer un thread producteur avec $count = 50$ et un thread consommateur, puis détacher le thread consommateur mais attendre la fin du producteur avant de se terminer.

Annexe

Signature par ordre alphabétique de certains appels système Posix utilisés dans l'UE.

```

int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
unsigned int alarm(unsigned int seconds);
int bind(int socket, const struct sockaddr *address, socklen_t address_len);
int close(int fd);
int dup(int oldfd);
int dup2(int oldfd, int newfd);
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
pid_t fork(void);
int fstat(int fd, struct stat *buf);
int ftruncate(int fd, off_t length);
void freeaddrinfo(struct addrinfo *res);
int getaddrinfo(const char *node, const char *service, const struct addrinfo *hints,
                struct addrinfo **res);
pid_t getpid(void);
pid_t getppid(void);
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
int kill(pid_t pid, int sig);
int listen(int sockfd, int backlog);
void *memcpy(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);
int mkfifo(const char *pathname, mode_t mode);
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
int munmap(void *addr, size_t length);
uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
int pipe(int pipefd[2]);
ssize_t read(int fd, void *buf, size_t count);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
int sem_destroy(sem_t *sem);
int sem_init(sem_t *sem, int pshared, unsigned int value);
sem_t *sem_open(const char *name, int oflag);
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
int sem_post(sem_t *sem);
int sem_wait(sem_t *sem);
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);
int shutdown(int socket, int how);
int sigsuspend(const sigset_t *mask);
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
int socket(int domain, int type, int protocol);
int stat(const char *path, struct stat *buf);
int unlink(const char *pathname);
pid_t wait(int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options);
ssize_t write(int fd, const void *buf, size_t count);

```